Homework Assignment No. 1

Sentiment Analysis of Product Reviews

Released: September 15, 2026 Total: 100 points

Due: October 13, 2026 at 2:59 PM (Pacific Time)

This assignment gives you hands-on experience with text representations and text classification for sentiment analysis. Imagine that an office-supplies retailer wants to understand customer feedback from product reviews. Prepare the reviews and compare three classifiers, then build a bigram language model and compare smoothing settings.

Submission. Submit one Python file, `sentiment.py`, to the HW1 assignment on Brightspace. You may use a notebook while developing your solution; no written report or notebook PDF is required for grading. Use the starter `sentiment.py` from the ZIP and implement its functions, specified on page 7. Submit your completed file as `sentiment.py`. You may add helper functions.

Run `python sentiment.py` to read `data.tsv` from the current folder and print the outputs on page 6. Use Python 3.12 and the versions in the supplied `requirements.txt`. Install NLTK's `stopwords` and `wordnet` resources before running locally. The grading environment will provide these resources; your submission must not download data or install packages while it runs. Importing your file must not read data, train models, or print output.

Complete the assignment individually and follow course policies. All graded items are checked through function results and script outputs. Hidden tests may use small inputs with the same specified format. Return computed values, not hard-coded results. There is no minimum accuracy or perplexity target.

1. Dataset Preparation (10 points)

Use the Amazon US **Office Products** reviews file from the original assignment's archive:

```
https://web.archive.org/web/20201127142707if_/https://s3.amazonaws.com/amazon-reviews-pds/t
sv/amazon_reviews_us_Office_Products_v1_00.tsv.gz
```

Extract the TSV file and name it `data.tsv`. If the archive is unavailable, contact course staff for the same file; do not substitute another category or version.

(a) **Read and inspect.** Implement `prepare_data`. Read the file with Pandas using tab separation, `dtype=str`, `quoting=csv.QUOTE_NONE`, `keep_default_na=False`, `on_bad_lines="skip"`, and `usecols=["review_body", "star_rating"]` in `read_csv`. Rename these two columns to `review` and `rating`. Convert ratings to numeric values; exclude missing/blank reviews and ratings outside the integers 1–5. Preserve the original review text and row order. Return the valid-data rating counts and number of excluded loaded rows.

(b) **Labels and sampling.** Map ratings 4–5 to positive (1) and ratings 1–2 to negative (0); discard neutral rating 3. Return and print positive, negative, and neutral counts before neutral removal or sampling. Sample 100,000 reviews per binary class without replacement, using `random_state=42` separately for each class.

(c) **Split.** Concatenate the positive sample followed by the negative sample and reset the index. Split with `test_size=0.2`, `stratify=labels`, and `random_state=42`. Preserve the returned row order and indices. Use these same 160,000 training and 40,000 test rows for all three classifiers. Split **before fitting** any vocabulary, IDF weights, or model. The function's sample-size argument supports smaller valid grading inputs; it does not change the required full-data experiment.

2. Data Cleaning (15 points)

Implement `clean_review(text)`. Apply the same function to every training and test review. To make the transformation reproducible, perform these operations in order:

1. Decode HTML entities with `html.unescape`, then lowercase the text.
2. Replace HTML tags matching `r"<[^>]*>"` with one space.
   Then replace URLs matching `r"https?://\S+|www\.\S+"` with one space.
3. Replace the left and right curly apostrophes (Unicode U+2018 and U+2019) with a straight apostrophe. Expand contractions using `contractions.fix(text)` with default options from the pinned library. For example, `won't` becomes `will not`. Do not substitute a custom contraction dictionary.
4. Replace characters matching `r"[^a-z\s]"` with spaces. Collapse all whitespace runs to one space and strip both ends.

Return the cleaned string. Preserve the raw and cleaned text for later stages. In the script, print average training-review character lengths before and after cleaning. Count spaces as characters and retain rows that become empty; their lengths are zero. Use all training rows in the averages.

3. Preprocessing (15 points)

Implement `preprocess_review(text)`, taking an already cleaned string:

(a) Split on whitespace. Remove tokens in NLTK's English stop-word list, **except** `no`, `nor`, and `not`; retain these three words.

(b) Apply NLTK's `WordNetLemmatizer.lemmatize` to each remaining token with the default noun setting. Do not use POS tagging or another lemmatizer.

(c) Join the resulting tokens with single spaces and return the string. Apply the function to both splits without dropping empty rows.

Print average training-review character lengths before and after this stage. "Before preprocessing" must equal "after cleaning." For a local check, process *"These pens aren't reliable."* through both functions and inspect the returned strings. No explanation or extra printed line is required.

4. Feature Extraction (10 points)

(a) Fit and transform. Implement `extract_features` using scikit-learn's `TfidfVectorizer` with default settings, including unigram features. For this assignment, use the specified scikit-learn version's default implementation, including `sublinear_tf=False`, IDF smoothing, and L2 normalization; classroom formula variants do not override these defaults. Fit it **only on the preprocessed training reviews** and transform both splits with that fitted vectorizer. Return the vectorizer and both sparse matrices. Print the training-matrix shape.

(b) Handle unseen and empty input. Keep the fitted vocabulary and IDF weights unchanged when transforming test reviews. A word occurring only in test reviews must not enter the vocabulary. A review containing only unknown words, or an empty review, must yield a zero row with the same number of columns as the training matrix. Use the returned feature matrices in Sections 5, 6, and 7; do not densify them.

Model Training and Evaluation

Scope for Sections 5 and 7. Perceptron and Naive Bayes have not been covered in class. Brief background explanations and references are provided below. You are only required to use the specified scikit-learn implementations; you do not need to implement the training algorithms from scratch or derive their mathematical formulas.

Textbook reference. Jacob Eisenstein, *Natural Language Processing*, Section 2.3.1 (Perceptron) and Section 2.2 (Naive Bayes):
`https://github.com/jacobeisenstein/gt-nlp-class/blob/master/notes/eisenstein-nlp-notes.pdf`

For Sections 5, 6, and 7, implement `train_model` and `evaluate_model`. Use the three scikit-learn classifiers below with the same training rows and sparse TF-IDF features. Leave unlisted parameters at their pinned-version defaults. Do not tune parameters, change thresholds, or increase iteration limits. Use `predict` for class labels.

For each model, return **Accuracy, Precision, Recall, and F1-score** on **both training and test data**. Use binary averaging, positive label 1, and `zero_division=0` for precision, recall, and F1. `evaluate_model` returns four unrounded scores on a 0–1 scale; the script prints all eight training/test scores to four decimal places.

Fit only on training data. Evaluation must not retrain or modify the supplied model. The grader may call these functions independently with small sparse matrices and fitted models.

5. Perceptron (10 points)

Background. The perceptron is a linear classifier that predicts a class using a weighted sum of the input features and a bias term. During training, it updates its weights when an example is misclassified, moving the decision boundary toward correctly classifying that example. In this assignment, the input features are TF-IDF vectors. We use the classic perceptron, which has no hidden layers and is different from a multilayer perceptron (MLP). For an introduction, see Eisenstein, Section 2.3.1. For implementation details, see the scikit-learn Perceptron documentation.

For model name `"Perceptron"`, train `sklearn.linear_model.Perceptron` with `random_state=42`, `max_iter=1000`, and `tol=1e-3`. Return the fitted model; have the script print its eight training/test metrics.

6. Logistic Regression (10 points)

For model name `"Logistic Regression"`, train `sklearn.linear_model.LogisticRegression` with `random_state=42`, `solver="lbfgs"`, `max_iter=1000`, and `tol=1e-4`. Return the fitted model; have the script print its eight training/test metrics.

7. Multinomial Naive Bayes (10 points)

Background. Naive Bayes classifies documents using Bayes' theorem and the assumption that features are conditionally independent given the class. Multinomial Naive Bayes estimates class-specific word probabilities from feature counts and combines them with class prior probabilities to make predictions. Although originally formulated for word counts, it is also commonly applied to nonnegative TF-IDF features, as in this assignment. The parameter `alpha=1.0` applies Laplace smoothing to avoid zero probabilities for features absent from a class's training examples. "Multinomial" describes the feature model; it does not require more than two output classes. See Eisenstein, Section 2.2 and the scikit-learn Multinomial Naive Bayes documentation.

For model name `"Naive Bayes"`, train `sklearn.naive_bayes.MultinomialNB` with `alpha=1.0` and other default settings on the same nonnegative TF-IDF features. Return the fitted model; have the script print its eight training/test metrics.

8. Bigram Language Model and Smoothing (20 points)

The retailer also wants to model common word sequences in customer feedback. Use a small subset of the **same review data** to explore why a language model needs smoothing. This exercise uses the n-gram material from Lectures 2–3 and does not add another sentiment classifier.

(a) Prepare sequences and counts (8 points). Within each existing split, skip reviews whose *cleaned* text is empty. Sample 5,000 training and 1,000 test reviews without replacement, using `random_state=42` for each sample. Use the cleaned text from Section 2, **before stop-word removal and lemmatization**. Split on whitespace; each review is one sequence.

Build a vocabulary of words occurring at least three times in this 5,000-review training subset. Map every other word to `<unk>` in both subsets. The prediction vocabulary V consists of these retained words plus `<unk>` and `</s>`, even if `<unk>` has zero training occurrences. Add one `<s>` at the start and one `</s>` at the end of each mapped review. The start token `<s>` is a context only and is *not* in V.

Implement bigram counts $C(h, w)$ using dictionaries or `collections.Counter`. Count transitions separately within each review, including the transition from `<s>` and the transition to `</s>`. Do not count transitions between reviews. Define

$$C(h) = \sum_{w \in V} C(h, w).$$

Return the vocabulary, bigram counts, and context counts from `fit_bigram`; print $|V|$ in the script. Do not fit this vocabulary on the test subset or reuse the TF-IDF vocabulary.

(b) Add-one smoothing and evaluation (7 points). On the mapped test sequences, count and print the number of predicted-token *occurrences* whose bigram has $C(h, w) = 0$. Count repeated occurrences separately, including transitions to `</s>`. These are transitions that an unsmoothed model cannot assign positive probability to from its training counts.

Implement add-one (Laplace) smoothing:

$$P_{+1}(w \mid h) = \frac{C(h, w) + 1}{C(h) + |V|}.$$

For a context never observed in training, all its counts are zero, so this formula still defines a probability distribution. Compute and print the **corpus-level test perplexity**:

$$\mathrm{PP} = \exp\left(-\frac{1}{N} \sum_{(h,w) \text{ in test transitions}} \log P_{+1}(w \mid h) \right).$$

Use natural logarithms. Here N is the total number of predicted tokens across all 1,000 test reviews, *including* one `</s>` per review and *excluding* `<s>`. Sum log probabilities over all reviews before dividing by N; do not average per-review perplexities. Labels are not used in this section, and no language-model library is needed.

Return the unseen-transition count, number N of predicted tokens, and unrounded perplexity from `evaluate_bigram`. This function must use the supplied model without updating its counts or vocabulary. The script prints the unseen-transition count and perplexity after the vocabulary size. No written interpretation is required.

8. Bigram Language Model and Smoothing (continued)

(c) Compare smoothing settings (5 points).

Extend part (b)'s `evaluate_bigram` with an optional argument `k=1.0`. Use add-k smoothing from Lecture 3:

$$P_{+k}(w \mid h) = \frac{C(h, w) + k}{C(h) + k|V|}, \qquad k > 0.$$

Compare $k = 1.0$, 0.1, and 0.01 on the same fixed model and test subset, using its corpus-level perplexity definition. Fit once; reuse the counts, vocabulary, and mapped reviews without resampling or refitting. An unseen context has probability $1/|V|$. The returned `unseen` and `n_tokens` counts must stay the same for every k.

Print $k = 1.0$ as part (b)'s `Bigram test perplexity`, followed by two extra perplexities for $k = 0.1$ and $k = 0.01$. Report all three values; no written interpretation or best-k selection is required. Do not select a final model from test results. No perplexity ordering is assumed.

Required Script Outputs

Print exactly **37 lines** to standard output, in the order below. Use the labels exactly as written. Angle-bracket fields below describe the required values; replace each field with your computed result. Counts and dimensions are integers. Length averages, metrics, and perplexity use four decimal places. Do not print sample reviews, explanations, setup messages, or extra blank lines. Return the detailed statistics through the specified functions.

1. Data and features (8 lines). The three class counts refer to the valid data before neutral-review removal and sampling. All four length averages use every training row (160,000 in the default run), including empty strings. The training-matrix column count is the number of TF-IDF features. The first dimension is 160000 for the full-data run and the actual training size when the grader uses a smaller sample.

```
Positive reviews: <integer>
Negative reviews: <integer>
Neutral reviews: <integer>
Average length before cleaning: <value>
Average length after cleaning: <value>
Average length before preprocessing: <value>
Average length after preprocessing: <value>
TF-IDF matrix shape: (<integer>, <integer>)
```

2. Classifier metrics (24 lines). Print the following eight-line block three times, replacing `<Model>` with, in order: `Perceptron`, `Logistic Regression`, and `Naive Bayes`. Each line contains one metric value.

```
<Model> Training Accuracy: <value>
<Model> Training Precision: <value>
<Model> Training Recall: <value>
<Model> Training F1-score: <value>
<Model> Testing Accuracy: <value>
<Model> Testing Precision: <value>
<Model> Testing Recall: <value>
<Model> Testing F1-score: <value>
```

3. Bigram model and smoothing comparison (5 lines). Use the Section 8 subsets and vocabulary. The first perplexity uses $k = 1.0$; the final two lines are part (c)'s comparison on the same model and test set.

```
Bigram vocabulary size: <integer>
Bigram unseen test transitions: <integer>
Bigram test perplexity: <value>
Bigram test perplexity (k=0.1): <value>
Bigram test perplexity (k=0.01): <value>
```

Before submitting: run your script from a fresh process and confirm that it emits all 37 lines. Confirm that importing it produces no output or training, that each required function can be called independently, and that no absolute file paths or runtime downloads are needed. The grader calls functions and parses your script output.

Section	1	2	3	4	5	6	7	8	Total
Points	10	15	15	10	10	10	10	20	**100**

Required Function Interface

Implement the following functions in `sentiment.py`. Functions must not print, except `run_experiment`. Return values at full precision; round only the printed outputs. Do not modify caller-owned inputs. Hidden tests obey the stated input conditions; arbitrary error handling is not required.

`prepare_data(path, n_per_class=100000)`
Return `(train_df, test_df, stats)`. Frames have exactly the columns `review`, `rating`, `label`; ratings and labels are integers. Keep indices from the reset, concatenated sample and preserve split order. `stats` is a dictionary with keys `rating_counts` (integer keys 1–5, including zero counts), `class_counts` (keys `positive`, `negative`, `neutral`), and `excluded_rows` (invalid loaded rows, counted once each). Counts refer to data before sampling. The grader supplies a valid TSV header, enough rows of each binary class, and `n_per_class` that is a positive multiple of 5. Match Pandas parsing with all Section 1 options, including the specified `usecols`.

`clean_review(text)` and `preprocess_review(text)`
Each takes one string and returns one string, following Sections 2 and 3. Empty input strings are valid. Preprocessing takes already cleaned text.

`extract_features(train_texts, test_texts)`
Return `(vectorizer, X_train, X_test)`. Inputs are ordered lists of preprocessed strings. Return a fitted `TfidfVectorizer` and sparse matrices with matching row order. Training inputs contain at least one eligible vocabulary token; both lists contain at least one review.

`train_model(name, X_train, y_train)`
Return the fitted classifier specified in Sections 5, 6, and 7. The name is one of the three exact model-name strings there. Inputs are a nonnegative sparse matrix and a corresponding 1-D label array containing both classes.

`evaluate_model(model, X, y)`
Return a dictionary with keys `accuracy`, `precision`, `recall`, `f1`. Inputs are a fitted classifier, a nonempty sparse matrix, and matching binary labels. A test case may contain only one class.

`fit_bigram(train_texts, min_count=3)`
Return a dictionary with keys `vocab`, `bigrams`, `contexts`. `vocab` is a set of strings including `<unk>` and `</s>` but not `<s>`. `bigrams` maps `(context, token)` tuples to positive integer counts; `contexts` maps contexts to positive integer totals. Omit zero-count entries; `dict` or `Counter` is acceptable. Inputs are nonempty lists of cleaned strings; the frequency threshold is a positive integer.

`evaluate_bigram(model, test_texts, k=1.0)`
Return a dictionary with keys `unseen` (integer), `n_tokens` (integer), and `perplexity` (float). Use add-k smoothing with the supplied positive finite real k; omitting it gives Section 8(b)'s add-one result. Use the model from `fit_bigram`. Test inputs are nonempty lists of cleaned strings. For either bigram function, an empty review string is the sequence `<s>` → `</s>`; the full-data subset selection still excludes empty cleaned reviews as specified in Section 8.

`run_experiment(path="data.tsv", n_per_class=100000, lm_train_size=5000, lm_test_size=1000)`
Run the full pipeline through the above functions, print exactly the 37 specified lines, and return `None`. The grader may supply smaller sample/subset sizes with sufficient nonempty cleaned reviews. Preserve the split order when sampling the bigram subsets. Do not cap, resample, or replace requested sizes. Under `if __name__ == "__main__":`, call this function with defaults; importing the module must not execute it.